



PONTIFICIA UNIVERSIDAD CATÓLICA DE CHILE
ESCUELA DE INGENIERÍA
DEPARTAMENTO DE CIENCIA DE LA COMPUTACIÓN

IIC2133 — Estructuras de Datos y Algoritmos 2'2025

Interrogación 2

7 de noviembre de 2025

Tiempo: 2 horas

Condiciones de entrega: Se deben entregar 3 de las 4 preguntas.

Forma de Entrega: La entrega es a través de una imagen digitalizada en Canvas. Al final de la prueba tienen 10 minutos para subir la imagen de la prueba y el torpedo, en formato vertical, cada pregunta por separado.

Dispositivo: Para digitalizar la prueba se permite el uso de UN dispositivo (teléfono, tablet) el cual debe estar guardado y en silencio durante la prueba.

Torpedo: Se permite una hoja tamaño oficio escrita a mano.

Evaluación: Cada pregunta tiene 6 puntos (+1 punto base). La nota de la interrogación es el promedio de las 3 preguntas entregadas

Salida de la sala: Si necesita salir de la sala al baño, debe dejar el celular adelante en la mesa con los ayudantes.

Lea todas las preguntas antes de iniciar las respuestas.

1. Hashing

Una planilla (hoja de cálculo) excel contiene un máximo de 1,048,576 filas por 16,384 columnas y 255 caracteres por celda, lo que supera largamente la memoria RAM de cualquier computador personal normal. Tomando en cuenta que lo usual es que las planillas sean mucho más pequeñas que el máximo, se usa una estructura llamada *matriz poco densa* que consiste en guardar solamente las casillas ocupadas. Si las columnas se llaman por una (o más) letras, las filas por un número y el contenido de una celda es un **string** de caracteres, responda:

- (2 pts.) Implemente la matriz poco densa usando tablas de hash; indique qué función de hash usaría. ¿Cómo se manejan las colisiones?
- (2 pts.) Implemente en forma eficiente y usando pseudocódigo, las funciones de buscar una celda (por fila y columna), insertar un nuevo valor o fórmula en una celda, borrar una celda. (Hint: las fórmulas hacen referencia a otras celdas.)
- (2 pts.) Calcule la complejidad de las funciones: Buscar, Insertar y Borrar y la fórmula `=suma(A1,A20)` que corresponde a la suma de los valores entre la celda A1 y A20.

(1pt) Una matriz excel contiene diferentes tipos de datos, por simplicidad se va a modelar como un texto por ejemplo de 255, independiente que sean string, números, fórmulas, fechas, etc. Para crear la tabla de hash se debe contar con el tamaño de esta; como ese dato no está lo vamos definir como N. En el caso de que el número de elementos sobrepase N se debe hacer un rehashing con un N mayor para así poder seguir guardando datos en la tabla (excepto colisiones por encadenamiento).

- Para la función de hash se debe mapear las coordenadas (fila, columna) de la celda como un solo valor. $h(c,f) = (h1(c)*x + f) \bmod N$ donde h1 es la transformación de la columna a un número, la columna

A=1..Z=26 contadas desde el abecedario más significativo. Las planillas excel usan tablas donde los datos están agrupados en un rectángulo, por lo que es esperable que los índices de filas y columnas sean seguidos. (1pt) Para las colisiones va a depender del factor de carga, todas las opciones vistas en clases son apropiadas. — Puede haber otras funciones de hash. La opción de manejo de colisiones debe ser consistente con los códigos más adelante. Para encontrar una celda, se debe almacenar sus coordenadas en el bucket (también usaremos como alternativa el valor de la celda, aunque no es lo mejor)

2.

```
función inicializarMatriz (M matriz, N) -- no se pide pero es útil para asegurar que los lugares va
  for i=1, N M[i]=null
```

```
(0,5 pt.)funcion buscarCelda(M matriz, C: columna, F:fila, v valor){-- de debe guardar el CF en el
  key = hash(C, F)
  if M[key]=v or M[key]=null return key
  else buscacolision(M[key],v) -- busca la celda en las colisiones se debe buscar las coordenadas
}
```

```
(0,5 pt.)funcion insertar(M matriz, C columna, F fila, v valor){
  key = hash(C, F)
  If buscarCelda(M, C, F)= null
    M[key] = valor
  Else
    M[key] = disponible(m[key]) -- en caso de colisión inserta en el primer lugar disponible de
  }
```

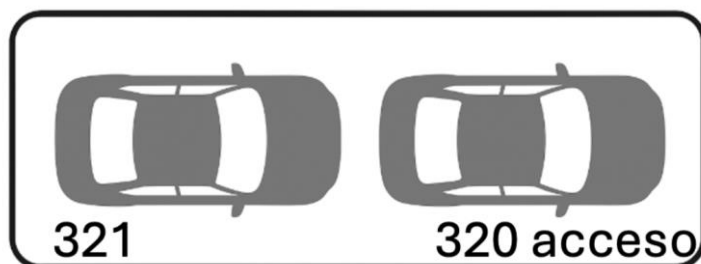
```
(0,5 pt.)funcion Borrar(M matriz, C columna, F fila, v valor){ -- se necesita el valor o almacenar
  If key<-buscarCelda(M, C, F) <> null
    If col<-buscarencolisión(key) <> null and M[key] <> v --busca la celda a borrar en la lista
      borracolisión(col) -- en caso de colisión borra el elemento de la lista
  }
```

(0,5 pt.) Por hacer las búsquedas en las colisiones

3. (1 pt.) En el mejor de los casos, es decir, sin colisión las funciones Buscar, Insertar, Borrar son $O(1)$. De lo contrario son $O(k)$ con k dependiendo de la lista de colisiones. (1 pt.) Para la evaluación de la fórmula $\text{suma}(A1..A20)$ se debe buscar TODOS los elementos entre $A1$ y $A20$ por separado por lo que es $20 \cdot O(1)$ en el mejor caso que en rigor es $O(1) \cdot 20 \cdot O(k)$ si hay colisiones. en general sería $F \cdot O(k) +$ la complejidad de la suma de pares $O(1)$. $F \cdot O(k)$.

2. Backtracking

En un subterráneo existen varios estacionamientos organizados en una configuración tándem, donde algunos espacios están dispuestos uno detrás de otro. Si dos estacionamientos (por ejemplo, 320 y 321) están en tándem, el vehículo que ocupe el espacio delantero (320) bloquea la entrada y salida del vehículo del espacio trasero (321). Esto implica que para que un vehículo pueda entrar o salir del espacio 321, el espacio 320 debe estar desocupado en ese momento. Existen también estacionamientos que no están en tándem, y los pares que están en tándem son siempre de a lo más dos espacios.



La administración del estacionamiento requiere asignar los N estacionamientos disponibles a M ($M \leq N$) vehículos. Para realizar esto, cada vehículo tiene asociada una hora de llegada y una hora de salida, la cual se debe considerar para que sea posible el acceso y salida desde los estacionamientos en tándem. Así, la administración debe determinar una asignación de vehículos a estacionamientos que cumpla las siguientes condiciones:

- Un mismo estacionamiento no puede ser asignado a más de un vehículo en intervalos de tiempo superpuestos.
- En los estacionamientos tándem, un vehículo no puede ser asignado al espacio trasero si durante su llegada o salida el espacio delantero estará ocupado.
- Todos los vehículos deben poder estacionar y retirarse sin bloqueos, respetando sus horarios.

En este contexto:

- a) (1 pt.) Modele este problema como un CSP, definiendo claramente variables, dominios y restricciones.

0,3 pts.	Variables: {Vehículo 1, ... ,Vehículo M}	o Estacionamientos
0,3 pts.	Dominios: {Estacionamiento 1, ... , Estacionamiento N}	o Vehículos
0,4 pts.	Restricciones: Cada Estacionamiento E_i tiene un atributo blockBy Cada Vehículo (V_j) tiene atributos hIn, hOut El Estacionamiento E_i puede ser asignado a V_j si: • E_i está libre en el intervalo $[V_j.hIn, V_j.hOut]$ & • $E_i.blockBy$ está libre en $V_j.hIn$ & en $V_j.hOut$	Puede ser “vacío” Llegada, Salida Puede ser descrito como <i>texto</i>

- b) (3 pts.) Proponga el pseudocódigo de la función `setParking()` para resolver la asignación de estacionamientos, indicando además los parámetros adecuados para realizar la llamada inicial.

2 pts.	<pre> setParking(V,E): // {vehículos},{estacionamientos} 1 if V=∅: return true 2 v ← un vehículo de V // el vehículo a asignar 3 for e ∈ E: 4 if test(v,e) : // si e puede ser asignado a v 5 v ← e // se asigna e a v 6 if setParking(V-{v},E): // intenta asignar el siguiente v 7 return true 8 v ← ∅ // se intenta nuevo e 9 return false // No hay solución válida </pre>
0,6 pt.	<pre> test(v,e): // vehículo, estacionamiento 1 if libre(e,v.hIn,v.hOut)&& libreBlock(e.blockBy,v.hIn,v.hOut): 2 return true // se puede asignar 3 return false </pre>
0,1 pt.	<pre> libre(e,hIn,hOut): 1 if e no tiene un vehiculo asignado en el intervalo (hIn,hOut): 2 return true // se puede asignar 3 return false </pre>
0,1 pt.	<pre> libreBlock(e,hIn,hOut): 1 if e no tiene un vehiculo asignado en hIn && hOut : 2 return true // se puede asignar 3 return false </pre>
0,2 pt.	Llamada inicial: <code>setParking({vehículos},{estacionamientos})</code>

- c) (2 pts.) Explique cómo aplicaría en su solución propagación, de modo de mejorar el desempeño de su algoritmo.

1,5 pt.	Al aplicar propagación se busca reducir el dominio de las variables disponibles (no asignadas) dada la asignación válida (cumple las restricciones) realizada, si la asignación falla, se debe des-hacer la propagación. En la función <code>setParking()</code> esto se realiza a continuación de las líneas 5 y 8 respectivamente.
0,5 pt.	Una opción de propagación es que al asignar válidamente a un vehículo <code>v</code> el estacionamiento <code>e</code> , cuando <code>e</code> es en tándem, se registren como utilizados en ese rango ambos estacionamientos.

3. Greedy y Programación Dinámica

En el puerto de San Antonio, una empresa logística prepara el plan de embarque de contenedores, en el buque “EDD”, hacia distintos destinos. El buque tiene una capacidad máxima de carga W (toneladas). Cada contenedor i tiene las siguientes características:

- Peso total (toneladas): w_i
- Valor comercial de la carga (USD): v_i
- Destino: puerto p_i , determina restricciones de seguridad para el contenedor.

Durante la planificación, se deben seleccionar los contenedores que serán embarcados de modo que:

- No se exceda la capacidad total W del buque.
- Se maximice el valor comercial total de la carga transportada.
- No se pueden fraccionar contenedores (no es posible embarcar parcialmente uno).

El jefe de operaciones propone usar un enfoque **codicioso (greedy)**:

“Ordenemos los contenedores según su relación valor/peso (v_i/w_i), de mayor a menor, y carguemos tantos como quepan, en ese orden.”.

Este enfoque prioriza la eficiencia económica por unidad de peso.

Responda:

- a) (1 pt.) Pruebe, mediante un contraejemplo, que la estrategia codiciosa propuesta no entrega necesariamente la carga óptima.

0,4 pt.	Capacidad buque $W = 10$; Contenedores: C_1 ($w_1 = 6$; $v_1 = 100$), C_2 ($w_2 = 5$; $v_2 = 60$) , C_3 ($w_3 = 5$; $v_3 = 60$); Orden: $C_1 = 100/6 = 16,6$; $C_2 = C_3 = 60/5 = 12$
0,6 pt.	Solución Codiciosa: Carga $\Rightarrow [C_1]$, Valor = 100, Peso = 6. Solución Óptima: Carga $\Rightarrow [C_2, C_3]$, Valor = 120, Peso = 10.

- b) (1 pt.) Modele el problema como un **CSP** (problema de satisfacción de restricciones) a resolver utilizando **programación dinámica**. Identifique las variables, sus dominios, las restricciones y la función a optimizar.

0,2 pt.	Variables: Contenedores, cada uno denotado como c_i .
0,2 pt.	Dominios: $c_i \in \{0, 1\}$ Define si el contenedor i es parte de la solución óptima.
0,3 pt.	Restricciones: La suma de los pesos de los contenedores incluidos en la solución óptima debe ser $\leq$ a W , la capacidad máxima de carga del buque.
0,3 pt.	Función por optimizar: La suma de $(c_i * v_i)$ para i de 1 a n (número de contenedores).

- c) (3 pts.) Describa como se estructuran los subproblemas a resolver y generalice este subproblema para el paso k de resolución del problema. Proponga una recurrencia para resolver el problema y el pseudo código de un algoritmo a partir de ella.

0,4 pt.	Si la solución óptima es $\Omega = \{C1, C2, \dots Cn\}$ entonces sabemos que hay una decisión binaria para C1, esto es C1 pertenece o no pertenece al óptimo. Esos subproblemas se pueden expresar como: $g_0(W) = \max\{g_1(W - w_1) + v_1, g_1(W)\}$
0,6 pt.	Generalizando para $0 \leq k < n$, Se obtiene la recurrencia $g_k(w) = \max\{g_{k+1}(W - w_{k+1}) + v_{k+1}, g_{k+1}(W)\}$ dónde: $g_n(w) = 0$ si $w \geq 0$; $g_n(w) = -\infty$ si $w < 0$;
2 pt.	<pre> opt(j, n, w): 1 if j = n : 2 if w < 0 : return -∞ 3 else : return 0 4 else: 5 if M[j] ≠ ∅ : 6 return M[j] 7 else: 8 M[j] ← max{v_{j+1} + opt(j+1, n, w-w_{j+1}), opt(j+1, n, w)} 9 return M[j] </pre>
0 pt.	La llamada inicial es $opt(0, n, W)$ n es el # de contenedores y W la carga máxima.

- d) (1 pt.) Considere que, por restricciones de seguridad, existen pares de contenedores (r,s) que no pueden viajar juntos en el buque. Describa brevemente cómo incluiría esta restricción en su solución anterior.

0,7 pt.	Lo que deseamos es que si $r=1$ (forma parte del óptimo), entonces $s=0$ (no forma parte del óptimo), y $s=1$, solo si $r=0$. Esto lo podemos incluir en la solución con parámetros en $opt()$ con los estados de inclusión de r y s en la solución $r = \{0,1\}$, $s = \{0,1\}$.
0,3 pt.	Así las condiciones en el código pueden ser similares a: <pre> ... if j = s && r = 1 : return -∞ ... if j = r - 1 : r = 1 con_r = opt(j+1, n, w-w_{j+1}) r = 0 sin_r = opt(j+1, n, w) M[j] ← max{v_{j+1} + con_r, sin_r} ... </pre>

Una nueva inteligencia artificial, shutupGPT, fue usada para diseñar el sentido del tráfico vehicular de las calles de una ciudad experimental, Ciudad Crepuscular, en las afueras de Santiago. Para asegurarse de que shutupGPT había hecho un buen trabajo, el Ministerio de Transporte decidió hacer un solo test: comprobar que el grafo correspondiente a los sentidos de las calles **no fuera acíclico**;

- a) Explica detallada y precisamente cómo se construye el grafo correspondiente a los sentidos de las calles. Es decir, cuáles son los vértices (nodos); cuáles son las aristas (arcos); y qué tipo de grafo es: con costos, sin costos, direccional, no direccional.

Las calles mismas constituyen las aristas (o arcos). Todo cruce de calles es un vértice (o nodo); por lo tanto, una arista es en realidad un tramo continuo de calle sin ningún cruce. Las aristas son direccionales, según el sentido del tráfico de la calle correspondiente; si una calle tiene tráfico en ambos sentidos, entonces se la representa mediante dos aristas, cada una dirigida en una de las dos direcciones. El grafo es direccional sin costos.

- b) Explica detallada y precisamente cómo se hace el test para determinar que el grafo no es acíclico, y por qué tiene sentido hacerlo.

Para determinar la presencia (o no) de ciclos en un grafo direccional, ejecutamos el algoritmo DFS; si durante la ejecución del algoritmo, llegamos a un vértice pintado de gris (o equivalentemente, identificamos una arista “hacia atrás”), entonces el grafo es cíclico.

Si el grafo direccional, que representa los sentidos del tráfico de las calles, fuera acíclico, entonces las personas que salen (en vehículo) de un lugar, no podrían volver a él.

La gente se fue a vivir a Ciudad Crepuscular y durante las primeras semanas todo funcionaba aparentemente bien. Sin embargo, después de un tiempo, algo curioso comenzó a ocurrir: se empezaron a formar “barrios”, de modo que la gente se movía (en vehículo) sólo dentro de sus respectivos barrios. Y después de algún tiempo adicional, algo verdaderamente extraño comenzó a ocurrir: algunos barrios empezaron a desocuparse, mientras que otros empezaron a sobrepoblarse. A base de lo estudiado sobre grafos en clases, responde:

- c) ¿Cómo se explica la formación de barrios?

Según lo estudiado en clases, lo que ocurre es que los sentidos de las calles fueron definidos de manera tal que si bien el grafo tiene ciclos, éstos son sólo “locales,” dando origen a lo que se conoce como componentes fuertemente conectadas (CFCs); en la práctica, cada CFC pasó a ser un barrio: dentro de cada CFC, es posible ir y volver entre cualquier par de vértices.

- d) ¿Cómo se explica la desocupación de ciertos barrios y el sobrepoblamiento de otros?
¿Qué tiene que hacer el Misterio de Transporte para revertir esta situación?

El grafo de las CFCs de un grafo direccional es un grafo direccional acíclico, de modo que si hay una forma de ir (en vehículo) de un vértice u que está en la CFC A a un vértice v que está en la CFC B , **no hay ninguna forma** de volver de v a u . En términos de la ciudad, lo que ocurrió fue que las personas empezaron a viajar a otros barrios (otras CFCs) y no pudieron volver.

Para cada calle que va de una CFC a otra (en un solo sentido), hay que agregar otra calle en el sentido inverso.